

用于其他功能。Linux 实现了一些不用于保存文件数据的文件系统，称为伪文件系统 (Pseudo File System)。表 11.4 给出了一些常见的 Linux 伪文件系统。

表 11.4 Linux 中的一些伪文件系统

名 称	常用挂载点	描 述
procfs	/proc	查看和操作与进程相关的信息和配置
sysfs	/sys	查看和操作与进程无关的系统配置
debugfs		用于内核状态的调试
cgroupfs		用于管理系统中的 cgroups
configfs	/sys/kernel/config	创建、管理和删除内核对象
hugetlbfs		查看和管理系统中的大页信息

通过使用文件的抽象，伪文件系统能够直接获得内核中 VFS 提供的命名、权限检查等功能。同时，由于文件接口的简单易用性，用户可以使用文件管理、查看、监控等工具与伪文件系统进行交互。伪文件系统的一个常见用途是允许用户态应用程序通过读取文件的方式读取内核提供的信息，并通过写入文件的方式对操作系统内核进行配置调整。例如，在输出记录 11.3 中，我们可以通过读取 /proc/filesystems 文件来查看当前 Linux 操作系统内核中支持的所有文件系统。其中以 nodev (no device) 开头的文件系统大多为伪文件系统，即并非直接从设备中保存和读取数据。其中 proc 正是提供 /proc/filesystem 这个文件接口的伪文件系统。

输出记录 11.3 使用伪文件系统中的文件查看所有支持的文件系统类型

[user@osbook ~] # cat /proc/filesystems
nodev sysfs
nodev tmpfs
nodev bdev
nodev proc
nodev cgroup
nodev cpuset
nodev devtmpfs
nodev configfs
nodev debugfs
nodev tracefs
nodev securityfs
nodev sockfs
nodev bpf
nodev hugetlbfs
nodev devpts
nodev autofs
nodev pstore
ext3
ext2
ext4

伪文件系统同样使用 VFS 定义的内存数据结构和方法。当伪文件系统中的文件被读取时，VFS 在进行路径解析后，会发现目标文件被该伪文件系统管理。因此，VFS 会通过记录内存数据结构中的文件系统方法，将文件请求交由该伪文件系统进行处理。之后，伪文件系统便可以根据文件名等特征进行特定的内核操作。如上述 `/proc/filesystems` 文件被读取时，`/proc` 伪文件系统会遍历保存在内存中的所有文件系统类型（`file_system_type` 结构），并将其逐一写入用户态传递的缓冲区中。同理，当伪文件系统中的文件被写入时，伪文件系统会根据请求类型对内核状态进行改变。

伪文件系统提供了一种在系统调用之外，应用程序和操作系统内核进行交互的方法，提高了操作系统的灵活性。同时由于用户态有大量的文件操作工具（如 `cat`、`echo`、重定向等）和脚本，系统管理员可以使用自己熟悉的工具，轻松地利用伪文件系统的文件接口检查和管理操作系统内核状态。

11.4 VFS 与缓存

在前面的讨论中，我们假设对文件系统中结构的修改都是直接在存储设备中进行的。然而，在存储设备上直接访问数据存在两个问题：首先，目前大多数存储设备的读写粒度为一个块（常见为 512B 或 4KB），而文件系统所进行的更改往往并非对齐到块的边界，其读写的字节数也并非恰好为块大小的整数倍；其次，存储设备的访问速度相比内存慢几个数量级，大量频繁的存储设备访问操作会成为应用程序的性能瓶颈。

11.4.1 访问粒度不一致问题和一些优化

文件系统使用内存作为中转来解决软件和硬件的访问粒度不一致问题。为了便于描述，我们假设一个存储设备的块大小为 4KB。当文件系统需要修改存储设备中的 8 字节时，需要先从存储设备中找到这 8 字节所在的块号，并将整个块的数据读入一个 4KB 的内存页中。文件系统在内存页中修改这 8 字节，并将修改后的内存页通过驱动写回存储设备中。这是一个“读取 - 修改 - 写回”的过程。

之所以要在修改和写回之前先读取，是因为我们只想修改存储块中的一部分，而写回操作会覆盖整个存储块。为了保证此存储块中其他部分的数据不变，需要先将这些数据读出来，之后随修改后的数据一同写回存储设备。一个明显的问题是：若每个文件请求中每个结构的修改都经过完整的“读取 - 修改 - 写回”，将会产生多次的存储设备访问。一些比较简单的优化可以避免不必要的存储设备访问。如果一次修改的数据量刚好覆盖了整个存储块，那么可以不用读取，直接将修改后的 4KB 数据写回存储设备中。此外，如果一个文件请求中的多次修改均是在同一个存储块中，那么可以将多次修改合并到一个“读取 - 修改 - 写回”过程中，即变为“读取 - 修改 1 - 修改 2 - 修改 3 - ... - 修改 n - 写回”。这样同样是访问了两次存储设备，却可以完成多次修改。

11.4.2 读缓存

文件的访问具有时间局部性，当文件的一部分被访问后，其有较高的概率会再次被使用。因此，当文件系统从设备中读取了某个文件的数据之后，可以让这些数据继续保留在内存中一段时间。这样，当应用程序再次需要读取这些数据时，就可以从此前保留在内存的数据中读取，从而避免了存储设备的访问。这便是文件系统中的读缓存（Read Cache）。

读缓存是需要占用内存空间的。为了防止读缓存占用过多的内存，操作系统会对读缓存的大小进行限制。当读缓存占用过多内存时，使用 LRU 等策略回收读缓存占用的内存。

11.4.3 写缓冲区与写合并

与读缓存相对应的是写缓冲区（Write Buffer）。在此前的介绍中，我们默认在一个写请求结束后，所有的写入数据均已经被持久化到存储设备。换句话说，如果在文件写入请求完成后立刻发生断电或崩溃等情况，在进行恢复之后，刚刚写入的文件数据依然能被读取。这是一个较强但十分合理的保证。然而由于存储设备的性能较差，若每个写请求均等待写入设备完成，文件写操作的延时将变长，吞吐量会严重下降，影响整个系统的性能。

为了实现更好的性能，在文件系统的设计中有一个权衡：在一个文件写请求返回应用程序之后，允许其修改的数据暂时没有被持久化到存储设备中。这种权衡允许文件系统暂时将被修改的数据保存在内存中，并在后台慢慢地持久化到存储设备上，然而这样却牺牲了一定的可靠性。在前面的例子中，如果在文件写入请求完成后立刻发生断电，再次开机之后，刚刚完成写入的数据可能会丢失。为了确保数据被持久化到设备中，POSIX 中规定了 `fsync` 接口，用于保证某个已打开文件的所有修改全部被持久化到存储设备中。

在这个权衡的前提下，我们此前的批量修改策略可以进一步拓展到跨请求的情况。当文件系统修改完文件数据后，其修改会被暂存在写缓冲区的内存页中。如果后续的文件请求需要读取或者修改相同存储块中的数据，文件系统可以直接在写缓冲区对应的内存页上进行读取或者修改。当可用内存不足，或者对应的 `fsync` 被调用时，写缓冲区内存页中的数据才被写回存储设备对应的存储块中。这样，一段时间内同一个存储块上的多个写请求可以合并成一个磁盘写操作。

11.4.4 页缓存

在 Linux 内核中，读缓存与写缓冲区的功能被合并起来管理，称为页缓存（Page Cache）。页缓存以内存页为单位，将存储设备中的存储位置映射到内存中。文件系统通过调用 VFS 提供的相应接口对页缓存进行操作。

当文件被读取时，文件系统会先检查其内容是否已经保存在页缓存中。如果存在，则文件系统直接从页缓存中读取数据并将其返回应用程序。如果不存在，文件系统会在页缓存中创建新的内存页，从存储设备中读取相关的数据，并将其保存在创建的内存页中。之后，文件系统从内存页中读取相应的数据，并将其返回应用程序。

在进行文件修改时，文件系统同样首先检查页缓存。如果要修改的数据已经在页缓存中，文件系统可以直接修改页缓存中的数据，并将该页标记为脏页。若不存在，文件系统同样先创建页缓存并从存储设备中读取数据，此后在页缓存中进行修改并标记为脏页。标记为脏页的缓存由文件系统定期写回存储设备中。在操作系统内存不足或者应用程序调用 `fsync` 时，文件系统也会将脏页中的数据进行写回。

图 11.21 给出了页缓存中内存页的状态和转移关系。文件系统从磁盘中读出数据，创建新的缓存页。此时缓存页为干净页，表示其中的数据与存储中的数据相同。当干净页中的数据被修改时，干净页被标记为脏页，表示其中的数据与磁盘中的数据有所区别。当页中的数据被写回存储设备后，页面再次变为干净页，可以被回收以释放空间。

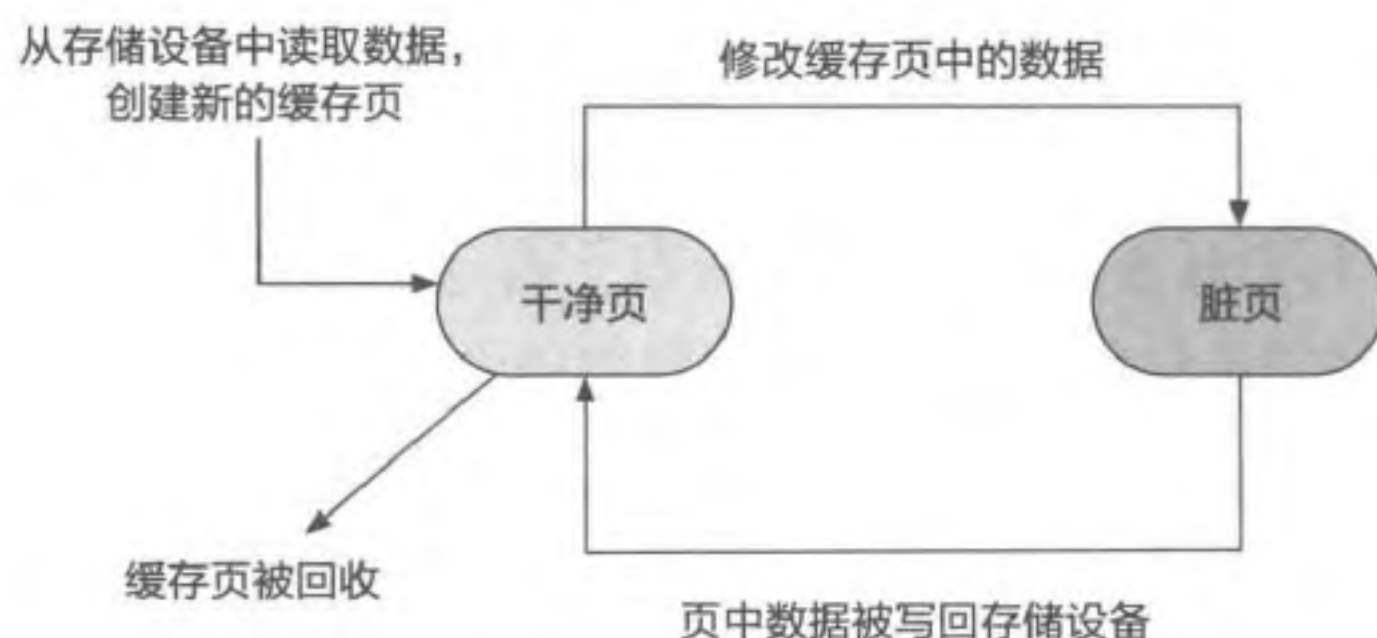


图 11.21 页缓存中内存页的状态和转移关系

小思考

页缓存一定能带来性能提升吗？

11.4.5 直接 I/O 和缓存 I/O

页缓存是持久化和性能之间的权衡。在大多数情况下，页缓存能够显著提升文件系统的性能，然而，在某些情况下，使用页缓存反而会起到负面作用。一种情况是，部分应用对数据持久化有较强的要求，不希望文件修改缓存在页缓存中。如果使用页缓存的机制，这些应用需要在每次修改文件后立即执行 `fsync` 操作进行同步，影响应用程序的性能。另一种情况是，一些应用程序（如数据库）自己实现缓存机制，以完全控制对数据的缓存和管理。由于应用程序更加了解自己对数据的需求，在这种情况下，操作系统提供的页缓存机制是冗余的，且一般会带来额外的性能开销。

因此，文件系统将是否使用页缓存的判断和选择权交给了应用程序。应用程序可以通过在打开文件时附带 `O_DIRECT` 标志，提示文件系统不要使用页缓存。这种文件访问方式就是直接 I/O (Direct I/O)，相对应的使用页缓存的文件请求被称为缓存 I/O (Cached I/O)。

11.4.6 内存映射

除了文件的 `read` 和 `write` 接口外，应用程序还可以通过内存映射机制以访问内存的形式访问文件内容。Linux 在其页缓存的基础上实现文件的内存映射机制。

在建立内存映射前，应用程序先打开目标文件，获得其文件描述符，随后调用 `mmap` 接口建立文件内存映射。调用时提供映射的目标虚拟地址^①、长度、属性、标志位、文件描述符和起始位置在文件中的偏移量。

在处理内存映射请求时，VFS 会分配对应的 VMA 结构，并通过在 VMA 结构中记录目标文件的 `inode` 和映射时的属性，将 VMA 对应的虚拟地址空间与文件 `inode` 进行关联，最后返回起始虚拟地址给应用程序。由于此时并未更新页表，当应用程序首次访问映射后的虚拟地址时，会触发缺页异常。Linux 在处理缺页异常时，会根据 VMA 中记录的 `inode` 信息，调用对应的文件系统进行处理。文件系统将从文件的页缓存中找到对应的内存页并返回给 VFS。VFS 最终将页缓存中内存页的物理地址填入页表，返回应用程序继续执行。由于页表中的映射已经建立，应用程序对同一个虚拟页的后续访问不会再触发缺页异常。

需要注意的是，在进行内存映射之后，应用程序在进行访问时，访问的是页缓存对应的内存页。其修改的数据保存在页缓存中，可能并未写回存储设备^②。因此，为了保证修改的数据写回存储设备上，应用程序需要调用 `msync` 接口请求 VFS 对指定内存映射区域进行同步写回操作。

当所有操作完成之后，应用程序可以调用 `munmap` 移除指定虚拟内存地址区域上的内存映射。

11.5 用户态文件系统

本节主要知识点

- ❑ 用户态文件系统有哪些优点？
- ❑ FUSE 是如何允许应用程序访问用户态的文件系统服务的？
- ❑ ChCore 是如何管理和协调多个文件系统的？
- ❑ ChCore 是如何实现一个简单的内存文件系统的？

① 需配合 `MAP_FIXED` 标志位使用，否则映射的目标虚拟地址由操作系统决定。

② 之所以说“可能”，是因为操作系统可能由于其他原因（如内存不足或其他应用调用了 `fsync`）已将部分页缓存中的数据修改写回存储设备。